

Test Results and Evaluation Report

SchEDU: Adaptive Academic Pacing Using Deterministic
Constraint-Based Greedy Scheduling and Heuristic Rebalancing

Software Engineering 2

Lyceum of the Philippines University

May 14, 2026

Contents

Revision Sheet	3
1 General Information	4
1.1 Purpose	4
1.2 Scope	4
1.3 System Overview	4
1.4 Project References	5
1.5 Acronyms and Abbreviations	5
1.6 Important Documentation Note	6
2 Test Analysis	7
2.1 Testing Objectives	7
2.2 Evidence Status Legend	7
2.3 Testing Approach and Levels	7
2.3.1 Static and Build Validation	7
2.3.2 Algorithm and Logic Testing	7
2.3.3 Manual Functional Testing	8
2.3.4 Backend and Data Validation	8
2.4 Test Environment	8
2.5 Security Considerations	8
3 Test Results and Evaluation	10
3.1 Static Validation Results	10
3.2 Scheduler Validation Matrix	10
3.3 Functional Test Case Matrix	11
3.4 Backend and Data Validation	13
3.5 Defect and Limitation Table	13

4	Summary and Conclusions	15
4.1	Demonstrated Capability	15
4.2	System Deficiencies	15
4.3	Recommended Improvements	15
4.4	System Acceptance	15

Revision Sheet

Release No.	Date	Revision Description
Rev. 1	May 14, 2026	Repository-tailored testing documentation drafted from the current IEEE paper, SRS, and source-backed validation evidence.

1 General Information

1.1 Purpose

This document records the testing evidence currently supported by the SchEDU repository and presents it in a defense-ready format. Its purpose is to verify that the implemented system behaves correctly, consistently, and safely under realistic academic planning use, while staying aligned with the current IEEE paper and Software Requirements Specification (SRS).

The report is intentionally evidence-based. It distinguishes between validation that was directly observed during repository review, validation that is strongly supported by the implementation wiring, and validation that remains conditional on local dependency cleanup or external service configuration. This prevents the document from overstating automation or maturity that the current repository does not yet demonstrate.

1.2 Scope

This report covers the implemented validation approach for the current SchEDU repository state as observed on May 14, 2026.

In scope

- Authentication and route protection behavior
- Profile, institution, subject, lesson plan, library, and calendar workflows
- Scheduler generation and rebalance logic
- Database-backed persistence for lesson plans, slots, blocks, events, and delays
- PDF extraction request flow
- AI-assisted activity generation request flow
- Static validation through linting

Out of scope or only partially covered

- Full automated UI test suites
- Large-scale performance or load benchmarking for production-scale usage
- Offline behavior validation
- Exhaustive cross-device compatibility coverage
- Mathematical optimization-score benchmarking, because the implemented scheduler is deterministic, rule-based, and validated through invariants rather than global objective scoring

1.3 System Overview

SchEDU is a teacher-facing curriculum and lesson planning workspace built with Expo Router and React Native, backed by Supabase authentication, database, storage, and edge functions. Its core differentiator is an adaptive scheduling engine that creates lesson-plan slots and blocks, then maintains schedule consistency when class availability changes.

- **Responsible organization:** Software Engineering 2 project team, Lyceum of the Philippines University
- **System name or title:** SchEDU: Adaptive Academic Pacing Using Deterministic Constraint-Based Greedy Scheduling and Heuristic Rebalancing
- **System code:** SchEDU
- **System category:** Major application
- **Operational status:** Under development; defense prototype
- **System environment:** Expo/React Native mobile frontend, Supabase backend, repository-local scheduler modules, and auxiliary OCR/AI services

From a testing perspective, the system has four practical layers:

1. Frontend navigation and user workflows under the Expo Router application.
2. Supabase-backed persistence for users, schools, sections, subjects, lesson plans, slots, and blocks.
3. Rule-based scheduler logic under the `algorithm/` directory.
4. Auxiliary edge functions for curriculum text extraction and activity generation.

1.4 Project References

1. IEEE paper: `papers/ieee/ieee.tex`
2. Software Requirements Specification: `papers/srs/srs.tex`
3. Repository overview: `README.md`
4. Project manifest and validation scripts: `package.json`
5. Scheduler invariant harness: `algorithm/__invariants.ts`
6. Scheduler wrapper script: `scripts/run-scheduler-invariants.sh`
7. Planning and scheduling schema: `database-setup/05_planning.sql`
8. Row-level security policies: `database-setup/06_rls.sql`
9. PDF extraction edge function: `supabase/functions/extract-text/index.ts`
10. Activity generation edge function: `supabase/functions/generate-activity/index.ts`

1.5 Acronyms and Abbreviations

Term	Meaning
API	Application Programming Interface
CI	Continuous Integration
OCR	Optical Character Recognition
POC	Point of Contact
RLS	Row-Level Security
SRS	Software Requirements Specification
UI	User Interface

UUID	Universally Unique Identifier
------	-------------------------------

1.6 Important Documentation Note

The SRS currently uses research-oriented terminology such as “constraint programming” and “simulated annealing.” The implemented repository behavior validated in this report is more accurately described as deterministic, rule-based, and rebalance-capable. This report therefore documents the testing evidence for the current implementation, not for a broader optimization concept that is not fully represented in code.

2 Test Analysis

2.1 Testing Objectives

The testing objectives for the current SchEDU implementation are the following:

- Confirm correctness of schedule generation and rebalance behavior.
- Confirm integrity of lesson-plan, slot, and block data structures.
- Confirm that critical teacher workflows execute without blocking design defects.
- Confirm access control and ownership safeguards in Supabase-backed flows.
- Confirm acceptable codebase stability through local static validation.

2.2 Evidence Status Legend

To avoid overstating evidence, the following status terms are used throughout this report:

- **Observed** — directly executed or verified during preparation of this report.
- **Repo-backed** — confirmed by implementation inspection and integration wiring, but not re-executed end-to-end on device during report preparation.
- **Conditional** — implemented, but dependent on missing local tooling, external service configuration, or native build prerequisites.
- **Limitation** — known gap or documentation issue that affects reproducibility or testing completeness.

2.3 Testing Approach and Levels

SchEDU currently follows a mixed validation approach. This is appropriate for the repository because the strongest automation exists in the scheduler layer, while the broader mobile application is validated through static checks, source-backed workflow tracing, and backend guard analysis.

2.3.1 Static and Build Validation

Static validation is currently anchored on `npm run lint`. This step catches unused variables, style issues, and obvious TypeScript or ESLint defects before runtime-oriented validation. On May 14, 2026, the lint command completed with warnings but no blocking errors.

2.3.2 Algorithm and Logic Testing

Algorithm validation is centered on `algorithm/__invariants.ts`. This harness checks initial plan generation, blackout handling, suspension and unsuspension rebalancing, no-data-loss conditions, idempotent maintenance behavior, and locked-block displacement plus re-pin handling. It is the strongest automated validation asset in the repository because it tests schedule consistency and recoverability rather than a purely visual workflow.

2.3.3 Manual Functional Testing

User-facing validation is documented as scenario-based testing across authentication, institution management, subject creation, library maintenance, lesson-plan creation, calendar loading, curriculum extraction, and activity generation. During preparation of this report, these scenarios were assessed primarily through repository-backed implementation tracing. Device-level walkthrough evidence may be appended later with screenshots or presenter notes for the final defense package.

2.3.4 Backend and Data Validation

Backend and data validation focuses on four concerns: schema constraints, row-level security, edge-function authentication, and ownership controls. This level does not yet constitute a separate automated integration test suite. Instead, it is documented as control verification against the existing schema and edge-function implementations.

2.4 Test Environment

Layer	Configuration	Purpose / Notes
Frontend	Expo Router, Expo SDK 54, React Native 0.81.5, React 19.1.0	Mobile user interface, navigation, and teacher workflows
Backend	Supabase Auth, Postgres, Storage, Edge Functions	Authentication, persistence, storage, and service endpoints
Scheduler	Repository-local modules under <code>algorithm/</code>	Slot creation, block construction, placement, ordering, and rebalance
Local command validation	<code>npm run lint</code> , <code>npm run test:scheduler</code>	Static validation and intended scheduler harness entrypoint
Observed workaround execution	Local TypeScript transpilation to <code>/private/tmp</code> plus Node execution	Used to run the invariant harness because the standard wrapper currently depends on missing <code>ts-node</code>

2.5 Security Considerations

Several security controls are directly visible in the repository and were considered during testing analysis:

- Supabase authentication is used to control access to protected app routes and authenticated backend operations.
- Row-level security is enabled for schools, subjects, lesson plans, slots, blocks, and related tables.

- The PDF extraction edge function requires a bearer token and rejects storage paths that do not belong to the authenticated user.
- The activity-generation edge function requires a bearer token and rejects requests missing required fields.
- Signed URLs are used when loading uploaded assets from storage, reducing direct public exposure of stored files.

At the same time, security testing remains implementation-centered rather than penetration-oriented. No full security audit, abuse-case suite, or production hardening certification is claimed by this report.

3 Test Results and Evaluation

3.1 Static Validation Results

Validation Item	Purpose	Observed Result	Status
<code>npm run lint</code>	Static validation of TypeScript and ESLint rules	Completed on May 14, 2026 with 7 warnings and 0 errors. The current codebase therefore has non-blocking cleanliness issues but no lint-level failures.	Observed
<code>npm run test:scheduler</code>	Standard repository entrypoint for scheduler invariants	The wrapper script points to <code>ts-node</code> , but the current repository dependencies do not include <code>ts-node</code> . The command is therefore not fully reproducible as wired.	Limitation
Invariant harness workaround execution	Execute the same scheduler harness without changing repository code	The harness was transpiled with the local TypeScript compiler and executed with Node. All invariant checks held successfully.	Observed

For traceability, the observed scheduler workaround executed the repository’s existing invariant source rather than a rewritten test. This preserves the core validation claim while also documenting the script-wrapper issue honestly.

3.2 Scheduler Validation Matrix

Scenario	Purpose	Expected Behavior	Evidence Source	Result
Initial plan generation	Confirm that the first build produces a coherent schedule	Lessons and required assessments are placed across generated slots with no lost blocks and coherent metrics.	<code>algorithm/__invariants.ts</code> plus observed workaround execution	Pass
Blackout handling	Confirm that unavailable days are recognized during planning	Slots affected by suspension or blackout conditions remain unavailable to normal placement logic.	Invariant harness and scheduler rebalance logic	Pass

Suspended-day rebalance	Confirm recoverable compression under reduced capacity	When one teaching day is suspended, remaining blocks are re-flowed without data loss.	Invariant harness, observed execution, rebalance modules	Pass
Unsuspend restoration	Confirm stability after availability returns	When the suspended day becomes available again, the layout returns to a stable configuration and can match the original arrangement.	Invariant harness, observed execution	Pass
Idempotent maintenance	Confirm that repeated maintenance does not introduce drift	Re-running rebalance on an already balanced schedule acts as a no-op.	Invariant harness, observed execution	Pass
Locked-block conflict handling	Confirm preservation of teacher intent during conflict	A teacher-locked block on a newly suspended day becomes displaced, remains visible, and receives suggested re-pin options instead of being discarded.	Invariant harness, observed execution	Pass

3.3 Functional Test Case Matrix

The following matrix records defense-relevant user workflow scenarios. Results are labeled according to the evidence-status legend so that repository-backed validation is not confused with full device automation.

ID	Module	Test Objective	Preconditions	Procedure	Expected Result	Actual Result	Status
AUTH-01	Authentication	Validate successful sign-in routing	Valid account credentials	Enter username or email and password, then sign in	User enters the protected tab flow after successful authentication	Password sign-in resolves email, calls Supabase sign-in, and routes to <code>/(tabs)</code> . Root session listeners support the same protected flow.	Repo-backed
AUTH-02	Authentication	Validate unauthenticated route protection	No active session	Open the app or a protected area without an authenticated session	User is redirected to the authentication flow	Root layout checks the current session and replaces unauthenticated access with <code>/(auth)</code> .	Repo-backed

PROF-01	Profile / Institution	Validate institution and section loading	Signed-in user with school membership	Open the institution screen for a linked school	Institution details and related sections are loaded from Supabase	The screen loads <code>user.schools</code> , school profile fields, signed avatar URLs, and related <code>sections</code> .	Repo-backed
SUBJ-01	Subject / Library	Validate subject creation with structured content persistence	Signed-in user and selected school	Create a subject, optionally add syllabus input, then save	Subject structure is persisted for later planning use	The flow supports syllabus text or asset handling and inserts subject, chapter, lesson, and <code>user.subjects</code> rows.	Repo-backed
LIB-01	Subject / Library	Validate editing of subject structure and lesson content	Existing subject and lessons	Open subject detail or lesson editor and update titles or content	Chapter, lesson, or lesson-content updates persist	Library screens issue update calls for structure fields and lesson HTML content.	Repo-backed
PLAN-01	Lesson Plan	Validate schedule generation and persistence	Existing subject, section, and lesson content	Complete the lesson-plan form and generate the plan	Plan, slots, blocks, subject-content links, and related events are created	The lesson-plan flow imports scheduler modules and persists plan rows, slot rows, block rows, content links, and related event rows.	Repo-backed
PLAN-02	Lesson Plan	Validate plan inspection and editing	Existing lesson plan	Open plan detail, inspect entries, then enter edit mode	Existing plan metadata and schedule series are loaded and can be updated	Plan detail loads the plan row plus slot rows, derives missing legacy series keys, and persists edits.	Repo-backed
CAL-01	Calendar / Scheduler	Validate sample fallback when no real plan is available	No teacher plan selected or load failure	Open the calendar screen	A non-empty demo schedule renders instead of a blank failure state	Calendar logic falls back to <code>DEMO.SCHEDULE</code> when no real plan is available or loading fails.	Repo-backed
CAL-02	Calendar / Scheduler	Validate rendering from persisted schedule rows	Persisted plan, slot, and block rows	Select a real lesson plan in the calendar view	Grouped schedule bars render from persisted planning rows	The calendar loader joins plan, slot, and block rows, groups split blocks, and labels them for display.	Repo-backed
OCR-01	Curriculum Extraction	Validate authenticated extraction request flow	Signed-in user, configured storage and extractor service	Upload a PDF or image and invoke extraction	The file is uploaded to the user's storage path and extracted text is returned	UI upload and function invocation are implemented; the edge function enforces bearer-token authentication and user-owned storage paths. Image OCR additionally requires a Dev Build.	Conditional

AI-01	Activity Generation	Validate guarded activity generation flow	Signed-in user and configured generation service	Select category, type, scope, and template, then generate	Backend validates inputs and returns a text draft for the activity	UI invocation is implemented; the edge function enforces authentication, required fields, and plain-text output. External AI configuration remains required.	Conditional
-------	---------------------	---	--	---	--	--	-------------

3.4 Backend and Data Validation

Area	Validation Focus	Observed Implementation Evidence	Status
Planning schema	Data integrity for lesson plans, slots, blocks, events, and delays	Foreign keys, date checks, time checks, unique constraints, category-subcategory pairing checks, indexes, and update triggers are defined in the planning schema.	Repo-backed
Row-level security	Access control over schools, subjects, plans, slots, and blocks	RLS is enabled across the planning domain, and policies are defined for own-school or own-user access patterns.	Repo-backed
PDF extraction edge function	Authenticated ownership-safe extraction requests	The function accepts only POST, requires a bearer token, validates the caller, limits extraction to <code>users/<uid>/</code> paths, and uses signed URLs.	Repo-backed
Activity generation edge function	Authenticated validated document generation requests	The function accepts only POST, requires a bearer token, rejects missing title/category/type fields, and returns text only when the AI response is non-empty.	Repo-backed

3.5 Defect and Limitation Table

Issue or Limitation	Impact	Current Workaround	Planned Improvement
---------------------	--------	--------------------	---------------------

Scheduler wrapper depends on missing <code>ts-node</code>	The documented <code>npm run test:scheduler</code> path is not fully reproducible as wired	Execute the same invariant source by transpiling with the installed TypeScript compiler, then run the emitted JavaScript with Node	Add <code>ts-node</code> to dev dependencies or replace the wrapper with a maintained local runner
Lint completes with warnings, not a fully clean report	Quality debt remains visible even though no blocking lint failures exist	Treat current lint results as acceptable for defense while recording the warnings transparently	Clean unused variables, array-style warnings, and BOM issues
No conventional Jest, Vitest, or UI automation suite	Broader app behavior is not covered by repeatable component or end-to-end automation	Use repository-backed scenario tracing and manual walkthrough evidence for defense	Introduce unit, integration, and UI automation suites with CI execution
Some documentation references are stale relative to current scheduler filenames	Readers may be confused when README descriptions no longer match active modules	Use repository paths from the current implementation in the testing report	Refresh README and related internal documentation to match current module names
SRS optimization terminology does not exactly match the implemented scheduler	Academic framing can overstate the algorithmic technique present in code	Distinguish proposed concept from implemented repository behavior during defense and in this report	Revise SRS wording so implementation claims consistently describe a rule-based, rebalance-capable scheduler

4 Summary and Conclusions

4.1 Demonstrated Capability

The current repository demonstrates a coherent academic planning prototype with a particularly strong scheduler-validation story. Static validation completes without blocking errors, the scheduler invariant harness covers the most failure-prone schedule-maintenance scenarios, and the surrounding application workflows are materially supported by implemented frontend, backend, and schema paths.

The most defensible implementation claim is therefore not that SchEDU already has a full production-grade automated test suite. Instead, the evidence shows that SchEDU has a mixed validation strategy: observed static validation, observed invariant-based scheduler testing, repository-backed user workflow coverage, and backend guard verification for sensitive flows such as extraction and activity generation.

4.2 System Deficiencies

The principal deficiencies identified during this report are:

- The standard scheduler test wrapper is not fully reproducible because `ts-node` is missing from the current dependency set.
- The broader mobile application does not yet have repeatable unit or end-to-end automation.
- Some supporting documentation still uses outdated scheduler module names or broader optimization terminology than the codebase supports.

These deficiencies do not negate the repository’s value as a defense implementation, but they do limit how strongly one can claim release readiness or full testing maturity.

4.3 Recommended Improvements

1. Restore a clean, reproducible scheduler test command by fixing the `test:scheduler` wrapper.
2. Add conventional unit and integration tests for frontend helpers, data mappers, and utility modules.
3. Introduce UI or workflow automation for high-value paths such as authentication, subject creation, lesson-plan generation, and calendar display.
4. Align README and SRS terminology with the repository’s actual deterministic and rule-based scheduler implementation.
5. Add defense appendices containing screenshots or presenter notes from device-level walkthroughs to complement the repository-backed evidence in this report.

4.4 System Acceptance

Based on the evidence documented here, SchEDU is acceptable for academic defense as an implemented and test-documented prototype. The repository demonstrates meaningful

functional capability, especially in scheduler behavior and protected backend workflows. However, the system should be presented as under development rather than as a fully release-tested production application.